Livraria API — guia do código (AV2)

O que cada arquivo faz, com o código real do projeto

Como ler este guia

O projeto é uma API REST em **Spring Boot 4.1** com **Java 25**. Este guia passa por cada parte do código, na ordem em que uma requisição atravessa o sistema, e mostra o código real com a explicação de cada trecho. O roteiro da demo e as perguntas prováveis estão em PDFs separados.

Estrutura de pastas

```
livraria-6P/
├── Dockerfile, .dockerignore, compose.yaml   ← Docker
├── pom.xml                                   ← dependências (Maven)
└── src/main/
    ├── resources/
    │   ├── application.yaml                  ← configuração (banco, Flyway, JWT)
    │   └── db/migration/V1..V5__*.sql        ← migrations do Flyway
    └── java/br/com/melqui/livraria/
        ├── config/       SecurityConfig, OpenApiConfig
        ├── security/     JwtService, JwtFilter
        ├── controller/   Livro, Autor, Categoria, Editora, Auth  (5 controllers)
        ├── service/      regra de negócio
        ├── repository/   acesso ao banco (Spring Data JPA)
        ├── model/        entidades (tabelas): Livro, Autor, Categoria, Editora, Usuario
        ├── dto/          formato de entrada (request) e saída (response) do JSON
        ├── mapper/       converte DTO ↔ entidade
        └── exception/    exceções e o GlobalExceptionHandler
```

O caminho de uma requisição

```
Cliente (Swagger / Postman)
   │  POST /api/v1/livros   Authorization: Bearer eyJ...   { "titulo": ..., "autor": "<id>", ... }
   ▼
JwtFilter ............ lê o token, valida, marca o usuário como autenticado
   ▼
Spring Security ...... rota exige login? usuário autenticado? se não → 401
   ▼
LivroController ...... recebe o JSON, valida o DTO (@Valid) → 400 se inválido
   ▼
LivroMapper .......... DTO → entidade Livro (campos simples)
   ▼
LivroService ......... busca autor/categoria/editora pelos IDs (404 se não existir) e salva
   ▼
LivroRepository ...... INSERT no banco (H2), tabelas criadas pelo Flyway
   ▼
LivroMapper .......... entidade → LivroResponseDto
   ▼
Resposta 201 Created + JSON do livro
(qualquer exceção no caminho → GlobalExceptionHandler → JSON de erro com o status certo)
```

1. Configuração

pom.xml — as dependências

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
        instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
        4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.1.1</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>br.com.melqui</groupId>
    <artifactId>livraria</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>25</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>3.1.1</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-flyway</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-h2console</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>
```

- **`spring-boot-starter-parent`**: define as versões de todas as bibliotecas do Spring, por isso a maioria das dependências não tem `<version>`.
- **`springdoc-openapi-starter-webmvc-ui`**: gera a documentação OpenAPI a partir dos controllers e a página do Swagger.

- **`spring-boot-starter-security`**: liga o Spring Security. Só de adicionar, todas as rotas ficam trancadas até o `SecurityConfig` dizer o contrário.
- **`jjwt-api`, `jjwt-impl`, `jjwt-jackson` (0.11.5)**: a biblioteca que cria e lê os tokens JWT. A `api` é usada no código; `impl` e `jackson` só em tempo de execução (`<scope>runtime</scope>`). O slide da aula usava `jjwt:0.11.5`, que não existe nessa versão.
- **`spring-boot-starter-flyway`**: roda as migrations ao subir a aplicação.
- **`spring-boot-starter-data-jpa`**: JPA/Hibernate (entidades e repositories).
- **`spring-boot-starter-validation`**: as anotações de validação (`@NotBlank`, `@NotNull`...).
- **`spring-boot-starter-webmvc`**: a parte web (controllers, Tomcat embutido).
- **`h2`** e **`spring-boot-h2console`**: o banco em memória.

application.yaml — a configuração

```yaml
server:
  port: 8080
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driverClassName: org.h2.Driver
    username: sa
    password: ''
  h2:
    console:
      enabled: true
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
    hibernate:
      ddl-auto: validate
    show-sql: true
  flyway:
    enabled: true
    locations: classpath:db/migration
jwt:
  secret: ${JWT_SECRET:bGl2cmFyaWEtNnAtY2hhdmUtc2VjcmV0YS1kZS1kZXNlbnZvbHZpbWVudG8tMjAyNg==}
  expiration: 3600000
```

- **`datasource`**: banco H2 em memória (`jdbc:h2:mem:testdb`). Ele existe só enquanto a aplicação roda: ao reiniciar, começa vazio.
- **`ddl-auto: validate`**: o Hibernate **não cria nem altera** tabelas; só confere se as entidades batem com o banco. Se faltar tabela ou coluna, a aplicação não sobe. Na AV1 era `update` (o Hibernate mexia no banco sozinho).
- **`flyway.enabled` / `locations`**: liga o Flyway e diz onde ficam os `.sql` (`src/main/resources/db/migration`).
- **`jwt.secret`**: a chave que assina os tokens. `${JWT_SECRET:...}` quer dizer "use a variável de ambiente `JWT_SECRET`; se não existir, use este valor padrão". Em produção a secret real vem do ambiente e nunca vai pro Git.
- **`jwt.expiration`**: validade do token em milissegundos (3.600.000 ms = 1 hora).
- O bloco `jwt:` fica **encostado na esquerda**: se ficasse dentro de `spring:`, a chave viraria `spring.jwt.secret` e o código não acharia `jwt.secret`.

2. Flyway — o banco versionado

Cada arquivo em `db/migration` é uma mudança no banco. O nome segue a regra **`V<versão>__<descrição>.sql`** (dois underscores). Ao subir, o Flyway compara a pasta com a tabela `flyway_schema_history` e aplica só as novas, em ordem. Migration aplicada **nunca se edita**: mudou algo, cria a próxima (V6...).

V1, V2, V3 — autor, categoria, editora

```
CREATE TABLE AUTOR (
  id UUID PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  nacionalidade VARCHAR(255)
);
```

```
CREATE TABLE CATEGORIA (
  id UUID PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  descricao VARCHAR(255)
);
```

```
CREATE TABLE EDITORA (
  id UUID PRIMARY KEY,
  nome VARCHAR(255) NOT NULL,
  cidade VARCHAR(255)
);
```

- **`id UUID PRIMARY KEY`**: o id é UUID e é a chave primária. O valor é gerado pelo Java (`@GeneratedValue`), não pelo banco.
- **`NOT NULL`**: obrigatório. Coluna sem `NOT NULL` (ex: `nacionalidade`) é opcional.
- Elas vêm antes do livro porque o livro aponta pra elas.

V4 — livro, com chaves estrangeiras

```
CREATE TABLE LIVRO (
    id UUID PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    isbn VARCHAR(255) NOT NULL,
    ano_publicacao INT NOT NULL,
    preco DECIMAL(10, 2) NOT NULL,
    autor_id UUID NOT NULL,
    FOREIGN KEY (autor_id) REFERENCES AUTOR(id),
    categoria_id UUID NOT NULL,
    FOREIGN KEY (categoria_id) REFERENCES CATEGORIA(id),
    editora_id UUID NOT NULL,
    FOREIGN KEY (editora_id) REFERENCES EDITORA(id)
)
```

- **`autor_id UUID NOT NULL`**: a coluna que guarda o id do autor do livro.
- **`FOREIGN KEY (autor_id) REFERENCES AUTOR(id)`**: a chave estrangeira. O banco **garante** que o id existe na tabela `autor`: não aceita livro com autor inexistente e não deixa apagar um autor que ainda tem livro (é daí que vem o 409).
- O mesmo para `categoria_id` e `editora_id`.

- `DECIMAL(10, 2)`: tipo pra dinheiro (até 10 dígitos, 2 depois da vírgula).
- `ano_publicacao`: o Hibernate converte o atributo `anoPublicacao` (camelCase) para `ano_publicacao` (snake_case).

V5 — usuário

```
CREATE TABLE USUARIO (
  id UUID PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL
);
```

- `email ... UNIQUE`: o banco não aceita dois usuários com o mesmo e-mail.
- `senha`: guarda o **hash BCrypt**, nunca a senha pura.

3. Model — as entidades

A entidade é a classe Java ligada a uma tabela: cada objeto é uma linha, cada atributo é uma coluna.

Livro

```
package br.com.melqui.livraria.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;

import java.math.BigDecimal;
import java.util.UUID;

@Entity
public class Livro {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String titulo;

    private String isbn;

    private Integer anoPublicacao;

    private BigDecimal preco;

    @ManyToOne
    @JoinColumn(name = "autor_id")
    private Autor autor;

    @ManyToOne
    @JoinColumn(name = "categoria_id")
    private Categoria categoria;

    @ManyToOne
    @JoinColumn(name = "editora_id")
    private Editora editora;
```

```java
    public UUID getId() {
        return id;
    }

    public void setId(UUID id) {
        this.id = id;
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getIsbn() {
        return isbn;
    }

    public void setIsbn(String isbn) {
        this.isbn = isbn;
    }

    public Integer getAnoPublicacao() {
        return anoPublicacao;
    }

    public void setAnoPublicacao(Integer anoPublicacao) {
        this.anoPublicacao = anoPublicacao;
    }

    public BigDecimal getPreco() {
        return preco;
    }

    public void setPreco(BigDecimal preco) {
        this.preco = preco;
    }

    public Autor getAutor() {
        return autor;
    }

    public void setAutor(Autor autor) {
        this.autor = autor;
    }

    public Categoria getCategoria() {
        return categoria;
    }

    public void setCategoria(Categoria categoria) {
        this.categoria = categoria;
    }

    public Editora getEditora() {
        return editora;
    }

    public void setEditora(Editora editora) {
        this.editora = editora;
    }
}
```

- `@Entity`: esta classe é uma tabela (`Livro` → `livro`).
- `@Id` + `@GeneratedValue(strategy = GenerationType.UUID)`: chave primária; o Hibernate gera o UUID antes do `INSERT`.
- `@ManyToOne`: o campo é uma **ligação com outra tabela**. *Muitos* livros para *um* autor. No banco fica só o id; no Java, o objeto `Autor` inteiro.
- `@JoinColumn(name = "autor_id")`: qual coluna da tabela `livro` guarda o id do autor (a mesma da V4).
- Ao **salvar**, o Hibernate pega `livro.getAutor().getId()` e grava em `autor_id`; ao **ler**, busca o autor por esse id e coloca o objeto no campo.
- Getters e setters: o Hibernate e o resto do código acessam os campos por eles.

Autor (Categoria e Editora seguem o mesmo molde)

```java
package br.com.melqui.livraria.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import java.util.UUID;

@Entity
public class Autor {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String nome;

    private String nacionalidade;


    public UUID getId() {
        return id;
    }

    public void setId(UUID id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getNacionalidade() {
        return nacionalidade;
    }

    public void setNacionalidade(String nacionalidade) {
        this.nacionalidade = nacionalidade;
    }
}
```

`Categoria` tem `nome` e `descricao`; `Editora` tem `nome` e `cidade`. O resto é igual.

Usuario

```java
package br.com.melqui.livraria.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import java.util.UUID;

@Entity
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    private String email;

    private String senha;

    public UUID getId() {
        return id;
    }

    public void setId(UUID id) {
        this.id = id;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getSenha() {
        return senha;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }
}
```

4. Repository — o acesso ao banco

```java
package br.com.melqui.livraria.repository;

import br.com.melqui.livraria.model.Livro;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.UUID;

@Repository
public interface LivroRepository extends JpaRepository<Livro, UUID> {

}
```

- É só uma **interface**: o Spring Data gera a implementação sozinho.
- `JpaRepository<Livro, UUID>`: "cuida da entidade `Livro`, cujo id é `UUID`". Vem pronto: `save`, `findById`, `findAll`, `delete`, `deleteById`, `existsById`...
- `AutorRepository`, `CategoriaRepository` e `EditoraRepository` são iguais, trocando a entidade.

```
package br.com.melqui.livraria.repository;

import br.com.melqui.livraria.model.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;
import java.util.UUID;

@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, UUID> {
    Optional<Usuario> findByEmail(String email);

    boolean existsByEmail(String email);
}
```

- **Métodos pelo nome (query methods)**: o Spring lê o nome e monta a consulta. `findByEmail` vira `SELECT ... WHERE email = ?`; `existsByEmail` devolve `true/false`.
- `Optional<Usuario>`: uma "caixa" que pode estar vazia (e-mail não encontrado) ou ter o usuário. Obriga o código a tratar o caso vazio.

5. DTOs — o formato do JSON

DTO (*Data Transfer Object*) é o formato de entrada e saída da API. Separa o que o cliente vê da entidade do banco. São `record`s: classes imutáveis, só com os campos.

LivroRequestDto — o que o cliente manda

```
package br.com.melqui.livraria.dto.request;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;
import jakarta.validation.constraints.Size;

import java.math.BigDecimal;
import java.util.UUID;

@Schema(description = "Dados para cadastrar ou atualizar um livro; autor, categoria e editora são
        os IDs já cadastrados")
public record LivroRequestDto(
        @Schema(example = "Dom Casmurro") @NotBlank @Size(max = 255) String titulo,
        @Schema(example = "3fa85f64-5717-4562-b3fc-2c963f66afa6") @NotNull UUID autor,
        @Schema(example = "3fa85f64-5717-4562-b3fc-2c963f66afa6") @NotNull UUID categoria,
        @Schema(example = "3fa85f64-5717-4562-b3fc-2c963f66afa6") @NotNull UUID editora,
        @Schema(example = "978-85-359-0277-8") @NotBlank @Size(max = 255) String isbn,
        @Schema(example = "1899") @NotNull @Positive Integer anoPublicacao,
        @Schema(example = "39.90") @NotNull @Positive BigDecimal preco) {
}
```

- Cada campo do record é um campo do JSON (`titulo` → `"titulo": "..."`).

- **`autor`, `categoria`, `editora` são `UUID`**: o cliente manda **só o id** de quem já está cadastrado, não o objeto inteiro.
- **Validação** (ativada pelo `@Valid` no controller; se falhar → 400):
  - `@NotBlank`: texto não pode ser vazio nem só espaços (só para `String`).
  - `@NotNull`: o campo tem que vir (qualquer tipo).
  - `@Size(max = 255)`: tamanho máximo do texto (bate com o `VARCHAR(255)`).
  - `@Positive`: número maior que zero.
- **`@Schema`**: documentação do Swagger: descrição e um exemplo que já aparece preenchido no "Try it out".

LivroResponseDto — o que a API devolve

```
package br.com.melqui.livraria.dto.response;

import java.util.UUID;
import java.math.BigDecimal;

public record LivroResponseDto(
        UUID id,
        String titulo,
        String autor,
        String categoria,
        String editora,
        String isbn,
        Integer anoPublicacao,
        BigDecimal preco) {
}
```

- A resposta mostra o **nome** do autor, da categoria e da editora (texto), em vez dos objetos.

DTOs de autenticação

```
package br.com.melqui.livraria.dto.request;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

@Schema(description = "Dados para criar uma conta")
public record RegisterRequestDto(
        @Schema(example = "usuario@email.com") @NotBlank @Email @Size(max = 255) String email,
        @Schema(example = "Senha@123") @NotBlank @Size(min = 6, max = 255) String senha) {
}
```

```
package br.com.melqui.livraria.dto.request;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.NotBlank;

@Schema(description = "Dados para fazer login")
public record LoginRequestDto(
        @Schema(example = "usuario@email.com") @NotBlank String email,
```

```
        @Schema(example = "Senha@123") @NotBlank String senha) {
}
```

```
package br.com.melqui.livraria.dto.response;

public record TokenResponseDto(String token) {
}
```

- `@Email`: valida o formato do e-mail. `@Size(min = 6)`: senha com pelo menos 6 caracteres.
- `TokenResponseDto` vira `{"token": "eyJ..."}` na resposta do login.

6. Mapper — DTO ↔ entidade

```
package br.com.melqui.livraria.mapper;

import br.com.melqui.livraria.dto.request.LivroRequestDto;
import br.com.melqui.livraria.dto.response.LivroResponseDto;
import br.com.melqui.livraria.model.Livro;
import org.springframework.stereotype.Component;

import java.util.List;

@Component
public class LivroMapper {

    public Livro toEntity(LivroRequestDto dto){
        Livro livro = new Livro();
        livro.setTitulo(dto.titulo());
        livro.setIsbn(dto.isbn());
        livro.setAnoPublicacao(dto.anoPublicacao());
        livro.setPreco(dto.preco());

        return  livro;
    }

    public LivroResponseDto toResponse(Livro livro){
        return new LivroResponseDto(
            livro.getId(),
            livro.getTitulo(),
            livro.getAutor().getNome(),
            livro.getCategoria().getNome(),
            livro.getEditora().getNome(),
            livro.getIsbn(),
            livro.getAnoPublicacao(),
            livro.getPreco()
        );
    }

    public List<LivroResponseDto> toResponseList(List<Livro> livros){
        return livros.stream().map(this::toResponse).toList();
    }
}
```

- `@Component`: o Spring cria uma instância e injeta onde precisar.
- `toEntity`: JSON → `Livro`. Copia só os campos simples. **Não** preenche autor/categoria/editora: transformar id em objeto exige ir ao banco, e isso é papel do service.
- `toResponse`: `Livro` → JSON. `livro.getAutor().getNome()` pega o nome de dentro do objeto. A ordem dos argumentos é a mesma dos campos do record.

- **`toResponseList`**: converte a lista inteira com `stream().map(...)`.

7. Service — a regra de negócio

LivroService

```java
package br.com.melqui.livraria.service;

import br.com.melqui.livraria.exception.LivroNaoEncontradoException;
import br.com.melqui.livraria.model.Livro;
import br.com.melqui.livraria.repository.LivroRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.UUID;

@Service
public class LivroService {
    private final LivroRepository livroRepository;
    private final AutorService autorService;
    private final CategoriaService categoriaService;
    private final EditoraService editoraService;

    public LivroService(LivroRepository livroRepository, AutorService autorService,
        CategoriaService categoriaService, EditoraService editoraService) {
        this.livroRepository = livroRepository;
        this.autorService = autorService;
        this.categoriaService = categoriaService;
        this.editoraService = editoraService;
    }
    public Livro criarLivro(Livro livro, UUID autorId, UUID categoriaId, UUID editoraId){
        livro.setAutor(autorService.buscarAutor(autorId));
        livro.setCategoria(categoriaService.buscarCategoria(categoriaId));
        livro.setEditora(editoraService.buscarEditora(editoraId));

        return livroRepository.save(livro);
    }

    public List<Livro> listarLivros(){
        return livroRepository.findAll();
    }

    public Livro buscarLivro(UUID id){
        return livroRepository.findById(id).orElseThrow(() -> new
        LivroNaoEncontradoException(id));
    }

    public Livro atualizarLivro(UUID id, Livro livro, UUID autorId, UUID categoriaId, UUID
        editoraId){
        Livro existente = buscarLivro(id);

        existente.setTitulo(livro.getTitulo());
        existente.setAutor(autorService.buscarAutor(autorId));
        existente.setCategoria(categoriaService.buscarCategoria(categoriaId));
        existente.setEditora(editoraService.buscarEditora(editoraId));
        existente.setIsbn(livro.getIsbn());
        existente.setAnoPublicacao(livro.getAnoPublicacao());
        existente.setPreco(livro.getPreco());

        return livroRepository.save(existente);
    }

    public void excluirLivro(UUID id){
```

```
        Livro existente = buscarLivro(id);
        livroRepository.delete(existente);
    }
}
```

- `@Service`: componente de regra de negócio, criado pelo Spring.
- **Construtor com 4 dependências**: injeção de dependência. O service pede ao Spring o repository e os services de autor, categoria e editora, e o Spring entrega prontos. Os campos são `final`: não mudam depois de criados.
- `criarLivro`: para cada id, chama o `buscar...` do service correspondente. Se o autor não existir, o `buscarAutor` **lança a exceção** e o método para ali (nada é salvo) → 404. Se existir, coloca no livro com `setAutor(...)` e salva.
- `buscarLivro`: `findById` devolve um `Optional`; o `orElseThrow` devolve o livro ou lança `LivroNaoEncontradoException`. Esse método nunca devolve `null`.
- `atualizarLivro`: busca o livro existente (404 se não houver), copia os campos simples do que chegou e busca de novo autor/categoria/editora pelos ids. O `set` vai no `existente`, que é o registro do banco.
- `excluirLivro`: busca antes de apagar. Sem isso, o `deleteById` com id inexistente não faria nada e a API responderia 204 mentindo; agora responde 404.

`AutorService`, `CategoriaService` e `EditoraService` têm os mesmos cinco métodos (criar, listar, buscar, atualizar, excluir), sem a parte das chaves estrangeiras.

AuthService — cadastro e login

```
package br.com.melqui.livraria.service;

import br.com.melqui.livraria.exception.CredenciaisInvalidasException;
import br.com.melqui.livraria.exception.EmailJaCadastradoException;
import br.com.melqui.livraria.model.Usuario;
import br.com.melqui.livraria.repository.UsuarioRepository;
import br.com.melqui.livraria.security.JwtService;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {
    private final UsuarioRepository usuarioRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public AuthService(UsuarioRepository usuarioRepository, PasswordEncoder passwordEncoder,
            JwtService jwtService) {
        this.usuarioRepository = usuarioRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    // cadastro: não deixa repetir e-mail e salva a senha embaralhada (BCrypt), nunca a senha pura
    public void registrar(String email, String senha) {
        if (usuarioRepository.existsByEmail(email)) {
            throw new EmailJaCadastradoException(email);
        }
        Usuario usuario = new Usuario();
        usuario.setEmail(email);
        usuario.setSenha(passwordEncoder.encode(senha));
        usuarioRepository.save(usuario);
    }
```

```
    // login: acha o usuário, confere a senha e, se estiver certa, devolve o token com o e-mail
        dele
    public String login(String email, String senha) {
        Usuario usuario = usuarioRepository.findByEmail(email)
                .orElseThrow(CredenciaisInvalidasException::new);
        if (!passwordEncoder.matches(senha, usuario.getSenha())) {
            throw new CredenciaisInvalidasException();
        }
        return jwtService.gerarToken(usuario.getEmail());
    }
}
```

- **`registrar`**:
  - `existsByEmail` → se já existe, lança `EmailJaCadastradoException` (409).
  - **`passwordEncoder.encode(senha)`**: transforma a senha num hash BCrypt antes de salvar. O BCrypt usa um *salt* aleatório: a mesma senha gera hashes diferentes, e não dá pra voltar do hash pra senha.
- **`login`**:
  - `findByEmail(...).orElseThrow(...)`: e-mail não existe → 401.
  - **`!passwordEncoder.matches(senha, hashDoBanco)`**: o `matches` aplica o BCrypt à senha digitada e compara com o hash salvo; o `!` faz lançar a exceção quando **não** bate → 401.
  - Mesma mensagem para e-mail ou senha errados ("E-mail ou senha inválidos"), pra não revelar quais e-mails existem.
  - Deu certo: **`jwtService.gerarToken(email)`** devolve o token. É aqui que o JWT nasce.

8. Controllers — as rotas

LivroController

```
package br.com.melqui.livraria.controller;

import br.com.melqui.livraria.dto.request.LivroRequestDto;
import br.com.melqui.livraria.dto.response.LivroResponseDto;
import br.com.melqui.livraria.mapper.LivroMapper;
import br.com.melqui.livraria.model.Livro;
import br.com.melqui.livraria.service.LivroService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;
import java.util.UUID;

@Tag(name = "Livros", description = "Cadastro de livros, ligados a autor, categoria e editora")
@RestController
@RequestMapping("/api/v1/livros")
public class LivroController {
    private final LivroService livroService;
    private final LivroMapper livroMapper;
```

```
    public LivroController(LivroService livroService, LivroMapper livroMapper) {
        this.livroService = livroService;
        this.livroMapper = livroMapper;
    }

    @Operation(summary = "Cadastra um livro")
    @ApiResponses({
            @ApiResponse(responseCode = "201", description = "Livro cadastrado"),
            @ApiResponse(responseCode = "400", description = "Dados inválidos"),
            @ApiResponse(responseCode = "401", description = "Sem token ou token inválido"),
            @ApiResponse(responseCode = "404", description = "Autor, categoria ou editora não
        encontrado")
    })
    @PostMapping
    public ResponseEntity<LivroResponseDto> cadastrarLivro(@RequestBody @Valid LivroRequestDto dto)
        {
        Livro livro = livroMapper.toEntity(dto);
        Livro  livroCriado = livroService.criarLivro(livro, dto.autor(), dto.categoria(),
        dto.editora());
        URI location = URI.create("/api/v1/livros/" + livroCriado.getId());
        return ResponseEntity.created(location).body(livroMapper.toResponse(livroCriado));
    }

    @Operation(summary = "Lista todos os livros")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Lista retornada"),
            @ApiResponse(responseCode = "401", description = "Sem token ou token inválido")
    })
    @GetMapping
    public ResponseEntity<List<LivroResponseDto>> listarLivros(){
        List<Livro> livros = livroService.listarLivros();
        return ResponseEntity.ok(livroMapper.toResponseList(livros));
    }

    @Operation(summary = "Busca um livro pelo ID")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Livro encontrado"),
            @ApiResponse(responseCode = "401", description = "Sem token ou token inválido"),
            @ApiResponse(responseCode = "404", description = "Livro não encontrado")
    })
    @GetMapping("/{id}")
    public ResponseEntity<LivroResponseDto> buscarLivro(@PathVariable UUID id){
        Livro livro = livroService.buscarLivro(id);
        return ResponseEntity.ok(livroMapper.toResponse(livro));
    }

    @Operation(summary = "Atualiza um livro")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Livro atualizado"),
            @ApiResponse(responseCode = "400", description = "Dados inválidos"),
            @ApiResponse(responseCode = "401", description = "Sem token ou token inválido"),
            @ApiResponse(responseCode = "404", description = "Livro não encontrado (ou autor,
        categoria ou editora não encontrado)")
    })
    @PutMapping("/{id}")
    public ResponseEntity<LivroResponseDto> atualizarLivro(@PathVariable UUID id, @RequestBody
        @Valid LivroRequestDto dto){
        Livro livro = livroMapper.toEntity(dto);
        Livro atualizado = livroService.atualizarLivro(id, livro, dto.autor(), dto.categoria(),
        dto.editora());
        return ResponseEntity.ok(livroMapper.toResponse(atualizado));
    }

    @Operation(summary = "Exclui um livro")
    @ApiResponses({
            @ApiResponse(responseCode = "204", description = "Livro excluído"),
            @ApiResponse(responseCode = "401", description = "Sem token ou token inválido"),
```

```
            @ApiResponse(responseCode = "404", description = "Livro não encontrado")
    })
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deletarLivro(@PathVariable UUID id){
        livroService.excluirLivro(id);
        return ResponseEntity.noContent().build();
    }
}
```

- `@RestController`: os métodos respondem JSON.
- `@RequestMapping("/api/v1/livros")`: prefixo de todas as rotas da classe.
- `@PostMapping` / `@GetMapping` / `@PutMapping` / `@DeleteMapping`: o verbo HTTP de cada método. `"/{id}"` + `@PathVariable` pega o id da URL.
- `@RequestBody @Valid LivroRequestDto dto`: converte o JSON no DTO e roda as validações (falhou → 400).
- `cadastrarLivro`: DTO → entidade (mapper), salva passando os ids (`dto.autor()`...), e responde **201 Created** com o cabeçalho `Location` apontando para o novo recurso.
- `ResponseEntity`: permite escolher o status: `ok` (200), `created` (201), `noContent` (204).
- **Anotações do Swagger**: `@Tag` agrupa as rotas na página; `@Operation` dá o título de cada rota; `@ApiResponses` lista os status possíveis e o que significam.
- `AutorController` (`/api/v1/autores`), `CategoriaController` (`/api/v1/categorias`) e `EditoraController` (`/api/v1/editoras`) têm as mesmas 5 rotas.

AuthController

```
package br.com.melqui.livraria.controller;

import br.com.melqui.livraria.dto.request.LoginRequestDto;
import br.com.melqui.livraria.dto.request.RegisterRequestDto;
import br.com.melqui.livraria.dto.response.TokenResponseDto;
import br.com.melqui.livraria.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import io.swagger.v3.oas.annotations.security.SecurityRequirements;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@Tag(name = "Autenticação", description = "Cadastro de usuário e login (gera o token JWT)")
@SecurityRequirements
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @Operation(summary = "Cadastra um usuário")
    @ApiResponses({
            @ApiResponse(responseCode = "201", description = "Usuário cadastrado"),
```

```
            @ApiResponse(responseCode = "400", description = "Dados inválidos"),
            @ApiResponse(responseCode = "409", description = "E-mail já cadastrado")
    })
    @PostMapping("/register")
    public ResponseEntity<Void> registrar(@RequestBody @Valid RegisterRequestDto dto) {
        authService.registrar(dto.email(), dto.senha());
        return ResponseEntity.status(HttpStatus.CREATED).build();
    }

    @Operation(summary = "Faz login e devolve o token JWT", description = "Copie o token da
        resposta e cole no botão Authorize para usar as outras rotas.")
    @ApiResponses({
            @ApiResponse(responseCode = "200", description = "Login feito; token na resposta"),
            @ApiResponse(responseCode = "400", description = "Dados inválidos"),
            @ApiResponse(responseCode = "401", description = "E-mail ou senha inválidos")
    })
    @PostMapping("/login")
    public ResponseEntity<TokenResponseDto> login(@RequestBody @Valid LoginRequestDto dto) {
        String token = authService.login(dto.email(), dto.senha());
        return ResponseEntity.ok(new TokenResponseDto(token));
    }
}
```

- **`/api/auth/register`** → 201 (sem corpo). **`/api/auth/login`** → 200 com `{"token": "..."}`.
- **`@SecurityRequirements`** vazio: marca essas rotas como públicas no Swagger (sem cadeado).
- São rotas liberadas no `SecurityConfig` (`/api/auth/**`), senão ninguém conseguiria fazer login.

9. Exceções — os erros com o status certo

```
package br.com.melqui.livraria.exception;

import java.util.UUID;

public class AutorNaoEncontradoException extends RuntimeException {
    public AutorNaoEncontradoException(UUID id) {
        super("Autor não encontrado: "+id);
    }
}
```

- Estende **`RuntimeException`**: não obriga `try/catch` em quem chama. O `super(...)` define a mensagem.
- Existem também `LivroNaoEncontradoException`, `CategoriaNaoEncontradaException`, `EditoraNaoEncontradaException`, `EmailJaCadastradoException` e `CredenciaisInvalidasException`.

```
package br.com.melqui.livraria.exception;

import br.com.melqui.livraria.dto.ErrorResponseDto;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;

import java.time.Instant;
import java.util.List;
```

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponseDto> handleGenerico(Exception ex, WebRequest request) {

        log.error("Erro inesperado", ex);

        return build(HttpStatus.INTERNAL_SERVER_ERROR, "Erro interno no servidor", request);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponseDto> handleValidacao(
            MethodArgumentNotValidException ex, WebRequest request) {

        List<String> erros = ex.getBindingResult().getFieldErrors().stream()
                .map(erro -> erro.getField() + ": " + erro.getDefaultMessage())
                .toList();

        return build(HttpStatus.BAD_REQUEST, String.join(", ", erros), request);
    }

    @ExceptionHandler(LivroNaoEncontradoException.class)
    public ResponseEntity<ErrorResponseDto> handleLivroNaoEncontrado(LivroNaoEncontradoException
        ex, WebRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(AutorNaoEncontradoException.class)
    public ResponseEntity<ErrorResponseDto> handleAutorNaoEncontrado(AutorNaoEncontradoException
        ex, WebRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(CategoriaNaoEncontradaException.class)
    public ResponseEntity<ErrorResponseDto>
        handleCategoriaNaoEncontrada(CategoriaNaoEncontradaException ex, WebRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(EditoraNaoEncontradaException.class)
    public ResponseEntity<ErrorResponseDto>
        handleEditoraNaoEncontrada(EditoraNaoEncontradaException ex, WebRequest request) {
        return build(HttpStatus.NOT_FOUND, ex.getMessage(), request);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponseDto> handleIntegridade(DataIntegrityViolationException ex,
        WebRequest request) {
        return build(HttpStatus.CONFLICT, "Não é possível concluir: existem registros vinculados",
        request);
    }

    @ExceptionHandler(EmailJaCadastradoException.class)
    public ResponseEntity<ErrorResponseDto> handleEmailJaCadastrado(EmailJaCadastradoException ex,
        WebRequest request) {
        return build(HttpStatus.CONFLICT, ex.getMessage(), request);
    }

    @ExceptionHandler(CredenciaisInvalidasException.class)
    public ResponseEntity<ErrorResponseDto>
        handleCredenciaisInvalidas(CredenciaisInvalidasException ex, WebRequest request) {
        return build(HttpStatus.UNAUTHORIZED, ex.getMessage(), request);
    }
```

```
    private ResponseEntity<ErrorResponseDto> build(HttpStatus status, String mensagem, WebRequest
        request) {
        ErrorResponseDto corpo = new ErrorResponseDto(
                Instant.now(),
                status.value(),
                status.getReasonPhrase(),
                mensagem,
                request.getDescription(false).replace("uri=", "")
        );

        return ResponseEntity.status(status).body(corpo);
    }
}
```

- `@RestControllerAdvice`: "vigia" todos os controllers. Quando uma exceção escapa, o Spring procura aqui o método com o `@ExceptionHandler` daquela exceção.
- Cada handler escolhe o status:

Exceção	Status	Quando
`MethodArgumentNotValidException`	400	DTO inválido (`@Valid`)
`...NaoEncontrad...Exception`	404	id não existe
`EmailJaCadastradoException`	409	e-mail repetido no cadastro
`DataIntegrityViolationException`	409	excluir algo que tem livro vinculado (chave estrangeira)
`CredenciaisInvalidasException`	401	login com e-mail ou senha errados
`Exception` (qualquer outra)	500	erro inesperado (e registra no log)

- `build(...)`: monta o `ErrorResponseDto` padrão (`timestamp`, `status`, `error`, `message`, `path`), então todo erro da API tem o mesmo formato.
- Assim o service só lança "não encontrado" e não precisa saber nada de HTTP.

10. Segurança — JWT

Como funciona, em resumo

1. O usuário faz **login** com e-mail e senha → recebe um **token**.
2. Em toda requisição seguinte, manda o token no cabeçalho: `Authorization: Bearer eyJ...`.
3. O `JwtFilter` confere o token; o **Spring Security** só deixa passar se ele for válido.
4. O servidor **não guarda sessão** (stateless): o token se prova sozinho.

O token tem três partes separadas por ponto, em Base64: **header** (algoritmo), **payload** (os dados: `sub` = e-mail, `iat` = criado em, `exp` = expira em) e **assinatura**. Base64 não é criptografia: qualquer um lê o payload. Mas a assinatura é calculada com a **secret**, que só o servidor tem; se alguém altera o payload, a assinatura não bate e dá 401.

JwtService — gera e valida o token

```java
package br.com.melqui.livraria.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.security.Key;
import java.util.Date;

@Service
public class JwtService {
    private final Key secret;
    private final long expiracao;

    // @Value lê o valor do application.yaml; a secret vira a chave que assina os tokens (slide 11)
    public JwtService(@Value("${jwt.secret}") String secret, @Value("${jwt.expiration}") long
        expiracao) {
        this.secret = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
        this.expiracao = expiracao;
    }

    // monta o crachá: dono (sub), quando foi criado (iat), quando vence (exp) e a assinatura
    public String gerarToken(String email) {
        Date agora = new Date();
        return Jwts.builder()
                .setSubject(email)
                .setIssuedAt(agora)
                .setExpiration(new Date(agora.getTime() + expiracao))
                .signWith(secret, SignatureAlgorithm.HS256)
                .compact();
    }

    // confere a assinatura e a validade; se o token foi adulterado ou venceu, lança exceção
    public String extrairEmail(String token) {
        return Jwts.parserBuilder()
                .setSigningKey(secret)
                .build()
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
    }
}
```

- **`@Value("${jwt.secret}")`**: lê o valor do `application.yaml`.
- **`Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret))`**: transforma a secret (texto em Base64) na chave usada pelo algoritmo HMAC-SHA256.
- **`gerarToken`**:
  - `setSubject(email)` → grava o e-mail no `sub` (o dono do token).
  - `setIssuedAt(agora)` → `iat`.
  - `setExpiration(agora + expiracao)` → `exp` (1 hora, vindo do yaml).
  - `signWith(chave, HS256)` → calcula a assinatura.
  - `compact()` → junta tudo na string `header.payload.assinatura`.
- **`extrairEmail`**:

- `parserBuilder().setSigningKey(chave).build()` → monta o leitor com a mesma chave.
- `parseClaimsJws(token)` → **recalcula a assinatura e confere a expiração**. Se o token foi alterado, venceu ou está malformado, **lança exceção**.
- `getBody().getSubject()` → lê o `sub`, o par do `setSubject`.

JwtFilter — o porteiro de cada requisição

```java
package br.com.melqui.livraria.security;

import io.jsonwebtoken.JwtException;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtFilter extends OncePerRequestFilter {
    private final JwtService jwtService;

    public JwtFilter(JwtService jwtService) {
        this.jwtService = jwtService;
    }

    // roda em toda requisição, antes do controller (slides 21 e 22)
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String jwt = extrairToken(request);
        if (jwt != null) {
            autenticar(jwt);
        }
        filterChain.doFilter(request, response);
    }

    // 1. extrai: o token vem no cabeçalho no formato "Bearer eyJhbGci..."
    private String extrairToken(HttpServletRequest request) {
        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            return header.substring(7);
        }
        return null;
    }

    // 2. valida e 3. autentica: se o token for válido, marca o usuário como logado nesta
        requisição
    private void autenticar(String jwt) {
        try {
            String email = jwtService.extrairEmail(jwt);
            UsernamePasswordAuthenticationToken autenticacao =
                    new UsernamePasswordAuthenticationToken(email, null, List.of());
            SecurityContextHolder.getContext().setAuthentication(autenticacao);
        } catch (JwtException | IllegalArgumentException e) {
            // token adulterado, vencido ou malformado: segue sem usuário, e o Security devolve 401
            SecurityContextHolder.clearContext();
        }
```

```
    }
}
```

- `extends OncePerRequestFilter`: base do Spring que garante que o filtro roda **uma vez por requisição**, antes do controller.
- `doFilterInternal`: extrai o token; se existir (`jwt != null`), tenta autenticar; **sempre** chama `filterChain.doFilter(...)` pra requisição seguir.
- `extrairToken`: lê o cabeçalho `Authorization`; se começar com `"Bearer "`, corta os 7 primeiros caracteres (`B-e-a-r-e-r-espaço`) e devolve só o token.
- `autenticar`:
  - `jwtService.extrairEmail(jwt)` valida e devolve o e-mail.
  - `new UsernamePasswordAuthenticationToken(email, null, List.of())` → o objeto que representa "usuário autenticado" (sem senha, sem papéis).
  - `SecurityContextHolder.getContext().setAuthentication(...)` → marca o usuário como logado **nesta requisição**.
  - Token inválido → `catch` limpa o contexto: segue sem usuário, e o Security devolve 401.
- O filtro **não barra** ninguém: só autentica ou não. Quem barra é a regra do `SecurityConfig`.

SecurityConfig — as regras

```
package br.com.melqui.livraria.config;

import br.com.melqui.livraria.security.JwtFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
public class SecurityConfig {
    private final JwtFilter jwtFilter;

    public SecurityConfig(JwtFilter jwtFilter) {
        this.jwtFilter = jwtFilter;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
                // API com token não usa cookie de sessão, então CSRF não se aplica
                .csrf(csrf -> csrf.disable())
                // sem sessão no servidor: cada requisição traz o próprio token
                .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                // quem pode passar: login/cadastro e Swagger abertos; o resto exige estar logado
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**").permitAll()
                        .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                        .anyRequest().authenticated())
                // sem login → 401
                .exceptionHandling(e -> e.authenticationEntryPoint(new
        HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
```

```
                // o JwtFilter roda antes do filtro padrão de usuário/senha
                .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
                .build();
    }

    // embaralha a senha antes de salvar (BCrypt); ninguém consegue ler a senha nem olhando o banco
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

- **`@Configuration` + `@Bean`**: a classe fornece objetos para o Spring usar (a cadeia de segurança e o `PasswordEncoder`).
- **`csrf.disable()`**: CSRF é um ataque que usa o cookie de sessão que o navegador manda sozinho. Esta API não usa cookie nem sessão (o token vai no cabeçalho), então não se aplica.
- **`SessionCreationPolicy.STATELESS`**: o servidor não cria sessão; cada requisição depende só do token.
- **`authorizeHttpRequests`**, avaliado de cima para baixo:
  - `/api/auth/**` → `permitAll()` (login e cadastro abertos).
  - `/swagger-ui/**`, `/v3/api-docs/**` → `permitAll()` (documentação aberta).
  - `anyRequest().authenticated()` → todo o resto exige usuário autenticado.
- **`HttpStatusEntryPoint(UNAUTHORIZED)`**: quem não estiver autenticado recebe **401** (sem isso o padrão seria 403).
- **`addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`**: coloca o `JwtFilter` antes do filtro padrão de usuário/senha, para o usuário já estar autenticado quando as regras forem checadas.
- **`passwordEncoder()` → `BCryptPasswordEncoder`**: usado no `AuthService` para gerar e conferir os hashes de senha.

11. Swagger — a documentação

```
package br.com.melqui.livraria.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI openAPI() {
        return new OpenAPI()
                .info(new Info()
                        .title("Livraria API")
                        .description("API para gerenciar livros, autores, categorias e editoras")
                        .version("1.0"))
                // botão Authorize: cola o token JWT e o Swagger manda "Authorization: Bearer ..."
        nas requisições
                .components(new Components()
```

```
                        .addSecuritySchemes("bearerAuth",
                                new SecurityScheme()
                                        .type(SecurityScheme.Type.HTTP)
                                        .scheme("bearer")
                                        .bearerFormat("JWT")))
                .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
    }
}
```

- `Info`: título, descrição e versão que aparecem no topo da página.
- `addSecuritySchemes("bearerAuth", ...)`: registra o esquema "Bearer JWT", que cria o botão **Authorize**.
- `addSecurityItem(...)`: aplica esse esquema a todas as rotas (as de auth tiram com `@SecurityRequirements`).
- Página: `http://localhost:8080/swagger-ui/index.html`. Especificação em JSON: `/v3/api-docs`.
- **OpenAPI** é a especificação (o formato que descreve a API); **Swagger** são as ferramentas (a página interativa). O **springdoc** gera a especificação lendo os controllers e as anotações.

12. Docker

Dockerfile

```
# Etapa 1: compilar o projeto (precisa do JDK e do Maven)
FROM eclipse-temurin:25-jdk-alpine AS build
WORKDIR /app

# copia só o que define as dependências e baixa tudo antes do código:
# assim o Docker reaproveita essa camada enquanto o pom.xml não mudar (cache)
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN chmod +x mvnw && ./mvnw -q dependency:go-offline

COPY src src
RUN ./mvnw -q package -DskipTests

# Etapa 2: rodar (só o JRE, imagem bem menor)
FROM eclipse-temurin:25-jre-alpine
ENV PORT=8080
WORKDIR /app

# não rodar como root: se algo der errado, o processo não tem privilégio de administrador
RUN adduser -D usuarioapp
USER usuarioapp

COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

- **Etapa 1 (`AS build`)**, com o JDK:
  - `FROM eclipse-temurin:25-jdk-alpine`: imagem base com Java 25 completo, versão alpine (pequena).
  - `WORKDIR /app`: pasta de trabalho dentro do container.
  - Copia **só** `mvnw`, `pom.xml` e `.mvn` e roda `dependency:go-offline` (baixa as dependências). Cada linha vira uma camada com cache: enquanto o `pom.xml` não mudar, essa camada é reaproveitada e o build não baixa tudo de novo.

    - Depois copia o `src` e roda `package` → gera o `.jar` em `target/`.

- **Etapa 2**, só para rodar:
    - `FROM eclipse-temurin:25-jre-alpine`: só o JRE, bem menor. A imagem final não leva compilador, Maven nem código-fonte.
    - `ENV PORT=8080`: variável de ambiente.
    - `adduser` + `USER usuarioapp`: roda com usuário comum, não root. Se alguém explorar uma falha, o processo não tem privilégio de administrador.
    - `COPY --from=build ... app.jar`: pega o `.jar` da etapa 1.
    - `EXPOSE 8080`: documenta a porta da aplicação.
    - `CMD ["java", "-jar", "app.jar"]`: o comando que roda quando o container sobe.

.dockerignore

```
target
.git
.idea
.vscode
.claude
aulas
*.md
Dockerfile
```

- Como o `.gitignore`: o que **não** entra na imagem. Menos arquivos = build mais rápido e imagem menor.

compose.yaml

```yaml
services:
  livraria:
    build: .
    image: livraria
    ports:
      - "8080:8080"
```

- **`build: .`**: monta a imagem a partir do `Dockerfile` desta pasta (= `docker build .`).
- **`image: livraria`**: nome da imagem (= `-t livraria`).
- **`ports: "8080:8080"`**: porta do computador : porta do container (= `-p 8080:8080`).
- Comando: **`sudo docker compose up --build`** (monta e sobe). Sem `--build`, usa a imagem já existente. Para parar: Ctrl+C.